# Preparcial: Sistema Evaluador de Riesgo Crediticio (Fintech)

Una plataforma de préstamos en línea (*Fintech*) recibe un CSV sobre el estado de aprobación de créditos de consumo. El sistema debe procesar un lote de solicitudes de clientes que se recibe para determinar estadisticas sobre la viabilidad de los préstamos. Este proyecto se entrega para estudiar y ejecutar; no hay tareas de implementación para esta instancia.

## Ejecución

Requiere JDK 21 o 25 y Maven 3.9.x. Desde esta carpeta:

```
mvn test
java -cp target/classes ar.edu.backend.fintech.Main
java -cp target/classes ar.edu.backend.fintech.Main datos/datos.csv
```

La primera ejecución de Maven puede requerir Internet. Una vez resueltas las dependencias, probar `mvn -o test` en la misma computadora. El programa no requiere dependencias externas en ejecución.

## Contrato del archivo

UTF-8, encabezado exacto `idCliente,tipoCliente,ingresosMensuales,totalDeudasActuales,estadoSolicitud`. Cada registro ocupa una línea con cinco campos separados por coma. Los campos no admiten comas, comillas ni saltos de línea internos; no hay campos entrecomillados. Se quitan espacios externos de cada campo. Es un formato de intercambio controlado, no un lector de todo CSV posible. El uso de `split(",", -1)` conserva los campos finales vacíos y es válido bajo este contrato.

Cada fila tiene id y tipo de Cliente no vacíos, ingresos mensuales y total de deuda actules enteros que representan miles de pesos, y estado de la solicitud. No se exige unicidad de identificadores del cliente; cada fila válida cuenta como un credito independientes independiente. Los tipos de clientes son textos sensibles a mayúsculas.

Los ingresos mensuales deben estar entre 1000 y 15000, el total de deuda tambien debe ser mayor a 0 pero no pueden ser iguales o superar a los ingresos mensuales. Estos cálculos y validaciones quedan en el objeto, en `SolicitudCredito.desdeCampos` y su constructor. El constructor recibe el **scoreFinaciero** de cada posible prestamo que manejó la *Fintech* se calcula como:

$$\text{scoreFinanciero} = \left( 1 - \frac{\text{totalDeudasActuales}}{\text{ingresosMensuales}} \right) \times 100$$

La Tasa de Interés Nominal Anual (TNA), a menor riesgo (mayor score), se le ofrece al cliente una tasa de interés más baja. El sistema calcula dinámicamente el interés que se le cobrará. Lógica del Cálculo: Se establece una tasa base del 40% y se le suma un *"recargo por riesgo"* inversamente proporcional al score, la fomula que se calcula ese **TNA** es:

$$\text{tasaInteresAnual} = 40 + (100 - \text{scoreFinanciero}) \times 0.8$$

## Flujo y errores

El parser verifica primero la cantidad de columnas. Con ancho correcto, `BLACK` se descarta sin validar los restantes datos. Para los estados que no sea `REGULAR` o `PREMIUM` se intenta convertir; cualquier otro

estado es inválido. Una línea vacía es inválida. No se interrumpe la carga por errores de una fila: se registra el número físico de línea y el motivo. Un encabezado incorrecto aborta con `IllegalArgumentException`; un fallo de lectura propaga `IOException`. El encabezado no cuenta como fila leída.

`procesadas` significa filas aceptadas, no intentos de conversión. Se cumple `leídas = procesadas + descartadas + inválidas` y `objetos = procesadas`. Se conserva el orden de los objetos aceptados y de los diagnósticos.

## Recorrido de lectura sugerido

1. Ejecutar `App` y los tres grupos de tests.
2. Leer `SolicitudCredito`: invariantes, conversión textual y tasa de interés derivado.
3. Seguir `ParserSolicitudes` y `ResultadoProceso`: clasificación y manejo de excepciones.
4. Leer `CentralCreditos`: copia defensiva, `Predicate<SolicitudCredito>`, filtros, suma y agrupamiento.
5. Localizar casos testigo en el archivo y explicar su clasificación y costo; usar el programa para verificar los agregados del conjunto.

El archivo contiene 60 filas de datos más encabezado (61 líneas). Panorama esperado: 60 leídas, 45 procesadas, 3 descartadas y 12 inválidas. Casos Testigos Aceptado: {'linea': 2, 'cliente': 'CL969', 'motivo': 'Cumple con todas las condiciones'}, Descarado: {'linea': 6, 'cliente': 'CL124', 'motivo': 'Tipo de cliente BLACK no permitido'}, Invalido: {'linea': 9, 'cliente': 'CL342', 'motivo': 'Deuda inválida o excede ingresos (6169)'}

Los getters se escriben explícitamente: no se agrega Lombok para minimizar dependencias y configuración del entorno de examen. ResultadoParseo también usa getters convencionales y conserva copias defensivas de sus listas. JUnit 5 es la única dependencia y se usa solamente en tests.